PROGRAMMING IN JAVA
ASSIGNMENT 4 (CO4)

EVENT-DRIVEN GUI DESIGN

Name: J. Subhaashree

Register Number: 192424008

Course Code: CSA09

Course: Programming in Java

1. Introduction

Event-driven programming organises an application around events generated by users or the operating system. In Java AWT/Swing, components such as JButton, JTextField and JComboBox generate event objects; registered listener objects receive callbacks on the Event Dispatch Thread (EDT). The EDT should execute short UI work only. Slow I/O, such as JDBC operations, should run in a worker thread, while Swing component updates return to the EDT. This assignment applies separation of concerns, layout managers, event listeners, repainting, validation, JDBC resource management and layered design.

2. Task 1 – GUI Design Review and Repaired Fee-Payment System

The original fee-payment design contains several issues. A single listener handles unrelated actions, coupling event routing and business logic. Comparing command strings with == compares references rather than text values. Amount input is not validated, so blank or malformed input can cause parsing errors and negative values may be accepted. JDBC calls and Thread.sleep on the EDT make the interface unresponsive. The Receipt action is not implemented distinctly. SQL string concatenation risks SQL injection and malformed queries. Connections and statements are not reliably closed. Exceptions and user feedback are not handled appropriately.

The repaired implementation separates FeeWindow (Swing view) from FeeService (validation and payment operation). FeeService validates the registration number and amount, rejects non-numeric, non-finite and non-positive amounts, and uses a PreparedStatement with parameter binding. Connection and statement objects are managed by try-with-resources. FeeWindow uses distinct actions for Pay, Clear and Receipt, displays status and dialogs, and performs payment through SwingWorker so JDBC does not block the EDT. The fee_payment.sql script creates the campus database and fees table.

Design defence: The view manages components and user interaction, while FeeService owns validation and payment rules. This separation makes the service independently testable and allows the interface to change without rewriting database logic. PreparedStatement reduces SQL-injection risk, try-with-resources closes JDBC resources, and SwingWorker preserves interface responsiveness during database operations.

3. Task 2 – Hospital Outpatient Appointment Desk

The event model maps each UI source to an event and listener. Department, doctor and time-slot combo boxes emit ItemEvent objects handled by ItemListener callbacks. The Book button emits ActionEvent and is handled by ActionListener. Window events are handled by WindowListener/WindowAdapter so closing can be controlled when a booking is in progress. The patient ID text field may use ActionListener to move focus to the next field.

The component hierarchy uses a JFrame with BorderLayout. The NORTH region contains a GridLayout form with patient ID, department, doctor and slot controls. The CENTER region is reserved for instructions or an appointment summary. The SOUTH region contains Book and Clear buttons in a FlowLayout panel.

Doctor refresh pseudocode: read the selected department; clear the doctor combo box; retrieve doctors for that department; add each doctor; if no doctors exist disable booking, otherwise select the first doctor and enable booking. Booking pseudocode: validate patient ID and all selections; disable the Book button while the booking is active; invoke the appointment service; show confirmation on success; restore button state and show a clear error on failure.

The event trace is: main → SwingUtilities.invokeLater → construct frame → initialise components and listeners → display frame → user event enters EDT → listener callback → service method → status/dialog update → window closing/closed event. Applet lifecycle methods init(), start(), paint(), stop() and destroy() are distinct from the JFrame lifecycle; a Swing application uses construction, visibility and window events.

A preloaded Map provides expected O(1) department lookup plus O(r) work to populate r doctors. Its storage is O(D+M), where D is department count and M is total doctor count. Querying a database on every department

change adds network and database latency and failure handling, but avoids retaining a full local doctor catalogue. Cache refresh is needed when doctor data changes.

4. Task 3 – Campus Library Search Screen

The library search interface uses a JFrame with BorderLayout. The NORTH area contains a FlowLayout search row with a text field, category combo box and Search button. The CENTER area contains a JTable inside a JScrollPane, allowing the result viewport to expand with the window. The SOUTH area contains a nested BorderLayout panel: action buttons in the CENTER and a status label in the SOUTH. The action buttons use FlowLayout.

The nested hierarchy is JFrame(BorderLayout) → search JPanel(FlowLayout) at NORTH; JScrollPane(JTable) at CENTER; south JPanel(BorderLayout) at SOUTH, containing actions JPanel(FlowLayout) at CENTER and status JLabel at SOUTH. BorderLayout gives the result table the available central space, while FlowLayout keeps controls at their preferred sizes.

Resize verification should be performed at 520×320, 760×480 and 1100×700. At each size, the table viewport should grow or shrink, and the search controls, buttons and status should remain visible. GridBagLayout offers flexible weighted constraints but requires more setup. Absolute positioning uses fixed coordinates and is not responsive; controls may overlap or leave gaps. For c components, a layout pass is approximately O(c). The fixed nested design uses O(1) panel containers.

5. Task 4 – Event-Handling Forensics and DrawPad

The callback name mouseclicked is incorrect because Java is case-sensitive and MouseAdapter defines mouseClicked with a capital C. Therefore, the misspelled method does not override the listener callback and is never invoked. A second defect is drawing directly through getGraphics(): the drawing is not retained as application state and can disappear after repainting, minimising or restoring the window.

The repaired DrawPad stores each mouse click as a Point in a list, updates the click-count label, calls repaint(), and redraws all stored points in paint(). Each point is rendered as a 6×6 oval centred at the click coordinate. The supplied MouseTypoCompileFail.java intentionally adds @Override to the misspelled callback; the expected compiler error proves that @Override detects the typo before runtime.

The point-list approach stores O(n) points and redraws O(n) points during a paint pass. An alternative is a BufferedImage canvas. Each click paints into the image and paint() copies the image to the screen. This uses approximately O(W×H) pixel storage (about 4WH bytes for ARGB) and avoids redrawing each point, though image allocation/resizing must be handled when the window dimensions change.

6. Task 5 – Currency Converter in Four Event-Handling Styles

Four implementations are provided. (1) The frame implements ActionListener and delegates actionPerformed to the conversion method. This is concise for a small example but combines the frame and listener roles. (2) A named inner Handler class implements ActionListener; it is reusable and can access the enclosing frame fields. (3) An anonymous inner class defines the listener at registration; it is explicit but can become verbose and is less reusable. (4) A lambda expression implements the single-method ActionListener interface concisely and is suitable for short callbacks.

All implementations accept an INR amount and a target currency. The demonstration conversion rates are fixed in the source: 1 INR = 0.012 USD and 1 INR = 0.011 EUR. The conversion logic formats the result to two decimal places and rejects malformed, negative or non-finite values. The same action is registered for the Convert button and Enter in the amount field.

Recorded console tests: 1000 INR → 12.00 USD; 1000 INR → 11.00 EUR; input abc → Invalid amount. These are demonstration rates embedded in the assignment code, not live market exchange rates. A lambda is shortest for a small handler; a named inner class is often clearer for substantial logic; a shared Action object can centralise enablement, icons, keyboard shortcuts and reuse across buttons and menus.

7. Task 6 – Campus Event Registration Application

The application is organised into four layers: EventRegistrationApp (Swing UI), RegistrationService (validation and use cases), RegistrationDAO (JDBC persistence), and Registration (data model). The SQL script creates campus_events.registrations with student ID, student name, event name, email and timestamp fields. A UNIQUE(student_id,event_name) constraint prevents duplicate registration for the same student and event, including concurrent attempts from separate application instances.

The UI includes a registration form, event selector, filter selector, JTable, status label and File menu actions for refresh, CSV export and exit. Required fields and email format are validated in RegistrationService. DAO uses PreparedStatement and try-with-resources for insert and paged select queries. SwingWorker runs database operations in doInBackground(); done() updates Swing components on the EDT. The table requests 20 rows per page. The application reads EVENT_DB_URL, EVENT_DB_USER and EVENT_DB_PASSWORD environment variables and requires MySQL Connector/J on the runtime classpath.

The application test plan covers valid registration, blank required fields, malformed email, duplicate registration, event filtering, page navigation and CSV export. The MySQL screenshots confirm that the campus_event_db database and its events and registrations tables are present.

Loading all N=100,000 records transfers and materialises O(N) rows and uses O(N) table-model memory. Paging P=20 records uses O(P) table-model memory and transfers O(P) rows per request. Database query work remains and suitable indexes help filtering and ordering. OFFSET pagination can become slower at large offsets; keyset pagination is a scalable alternative.

8. Testing and Execution Evidence

The uploaded execution log reports successful compilation of ten targets: FeeWindow, AppointmentDesk, LibrarySearch, DrawPad, four converter GUI variants, ConverterLogic and EventRegistrationApp. The appointment service console test lists two Cardiology doctors, confirms appointment P1001 with Dr Rao at 10:30, and rejects a blank patient ID. Converter logic tests return 12.00 USD and 11.00 EUR for 1000 INR and reject abc.

The intentionally failing MouseTypoCompileFail.java test is a compile-time diagnostic exercise. It is expected to fail because @Override is applied to a method with an incorrect callback spelling. It must not be counted as a failure of DrawPad.java.

The test_cases.csv file records the task-level test matrix. The screenshot appendix provides desktop evidence for the fee-payment window, library search, DrawPad, currency conversion and MySQL database/table setup, together with the appointment service console test.

9. Conclusion

The assignment demonstrates Java event-driven GUI design through Swing components, listener callbacks, layout managers, persistent painting and layered application structure. The included source code, console tests and desktop screenshot appendix document the implementations and observed outputs. The design uses validation, PreparedStatement, try-with-resources and SwingWorker to improve correctness, resource safety and responsiveness.

Appendix A – Compilation Summary

Task	Source file	Recorded status
Q1	FeeWindow.java	Compiled
Q2	AppointmentDesk.java	Compiled
Q3	LibrarySearch.java	Compiled
Q4	DrawPad.java	Compiled
Q5	ConverterFrameListener.java; ConverterNamedInner.java; ConverterAnonymous.java; Converter.java; ConverterLogic.java	Compiled
Q6	EventRegistrationApp.java	Compiled

Appendix B – Recorded Console Outputs

Appointment desk

Cardiology doctors: [Dr Rao, Dr Iyer]
Appointment confirmed: P1001 | Cardiology | Dr Rao | 10:30
Validation: Patient ID is required

Currency conversion

1000 INR to USD: 12.00 USD
1000 INR to EUR: 11.00 EUR
Invalid input: Invalid amount

Appendix C – Test Case Matrix

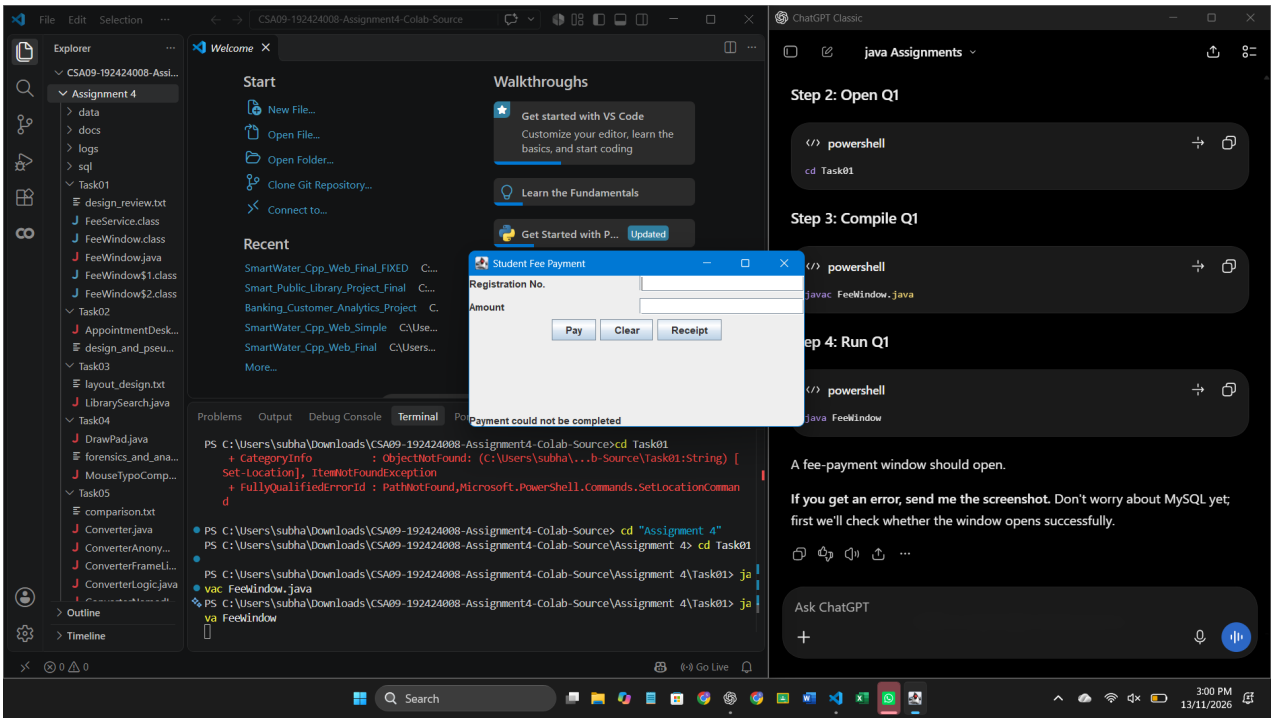
Task	Test	Input/interaction	Expected result	Verification in notebook
1	T1	blank registration number	validation error	design + FeeService source
1	T2	non-numeric/ negative amount	validation error	design + FeeService source
2	T1	Cardiology department	Cardiology doctor list	AppointmentDesk console
2	T2	blank patient ID	required-field error	AppointmentDesk console
3	T1	search term entered	results table populated	Swing source compiled
3	T2	resize 520x320, 760x480, 1100x700	centre results area resizes	desktop screenshot required
4	T1	click at point	count increments and point stored	source compiled
4	T2	minimise and restore	stored points redraw	source compiled; desktop screenshot

				required
5	T1	1000 INR to USD	12.00 USD	ConverterLogic console
5	T2	1000 INR to EUR	11.00 EUR	ConverterLogic console
5	T3	abc amount	Invalid amount	ConverterLogic console
6	T1	valid unique registration	row inserted	requires MySQL runtime
6	T2	blank/invalid email	validation error	service source compiled
6	T3	same student/event twice	unique constraint rejects duplicate	SQL constraint included; requires MySQL runtime

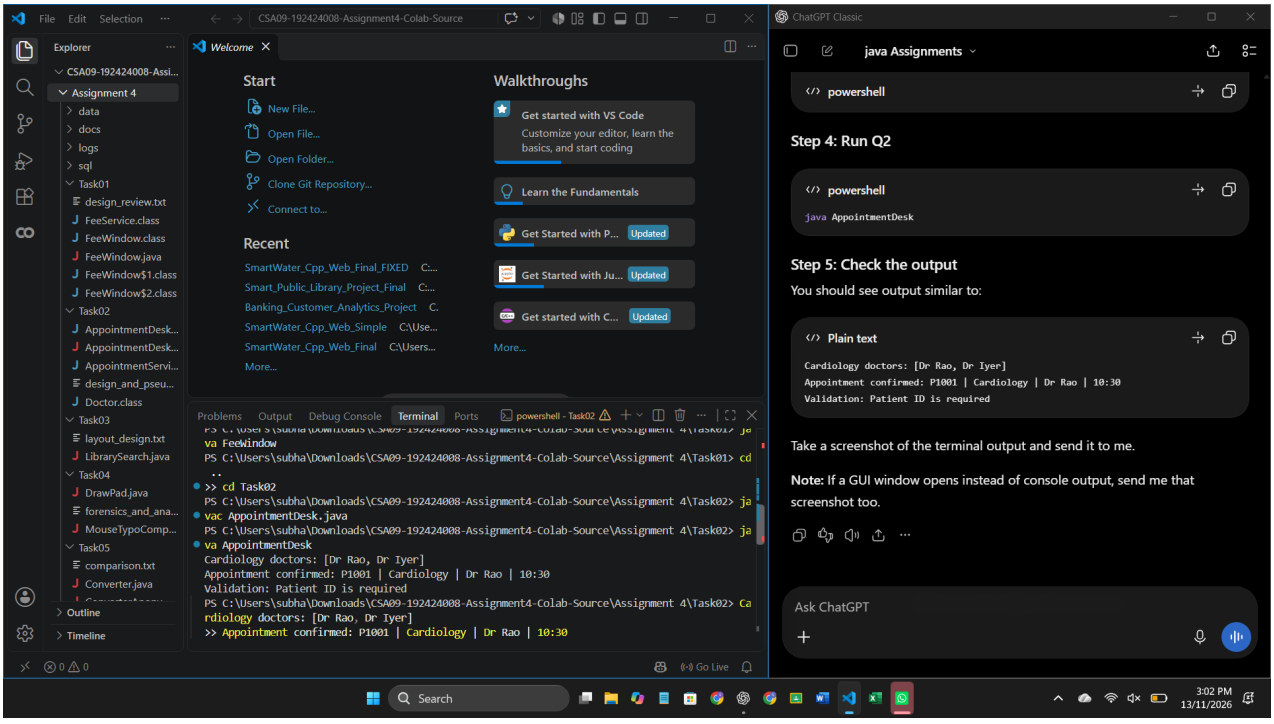
APPENDIX D – DESKTOP SCREENSHOT EVIDENCE

The following screenshots were captured from the desktop execution session and are included as visual evidence for the assignment tasks.

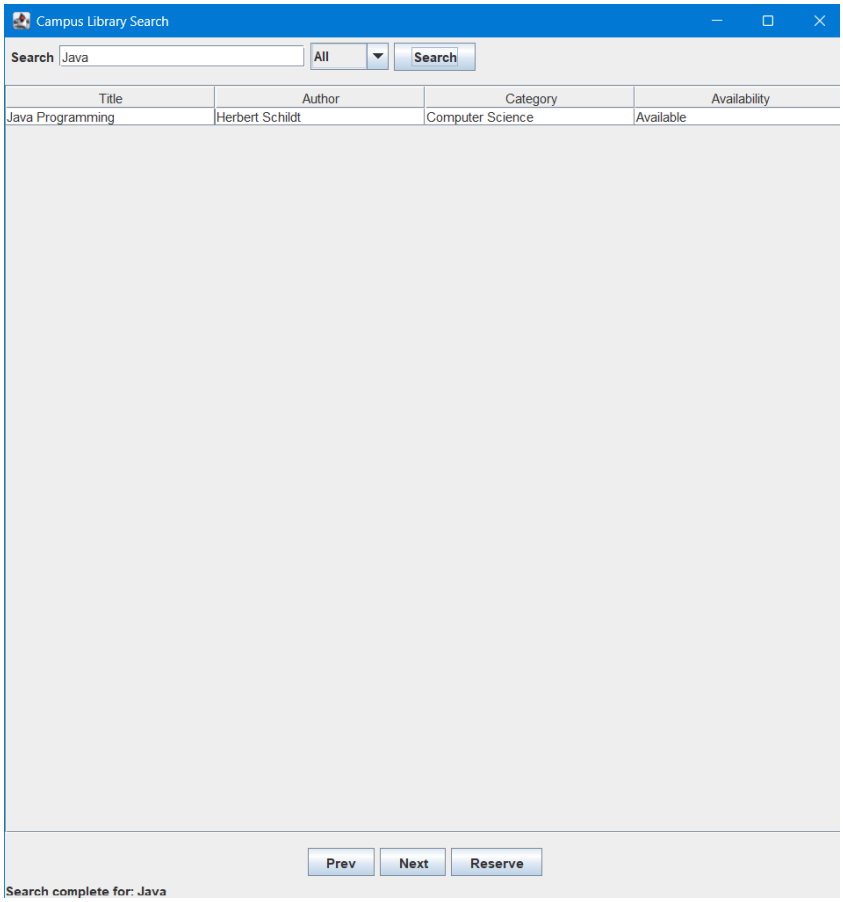
Task 1 – Student Fee Payment



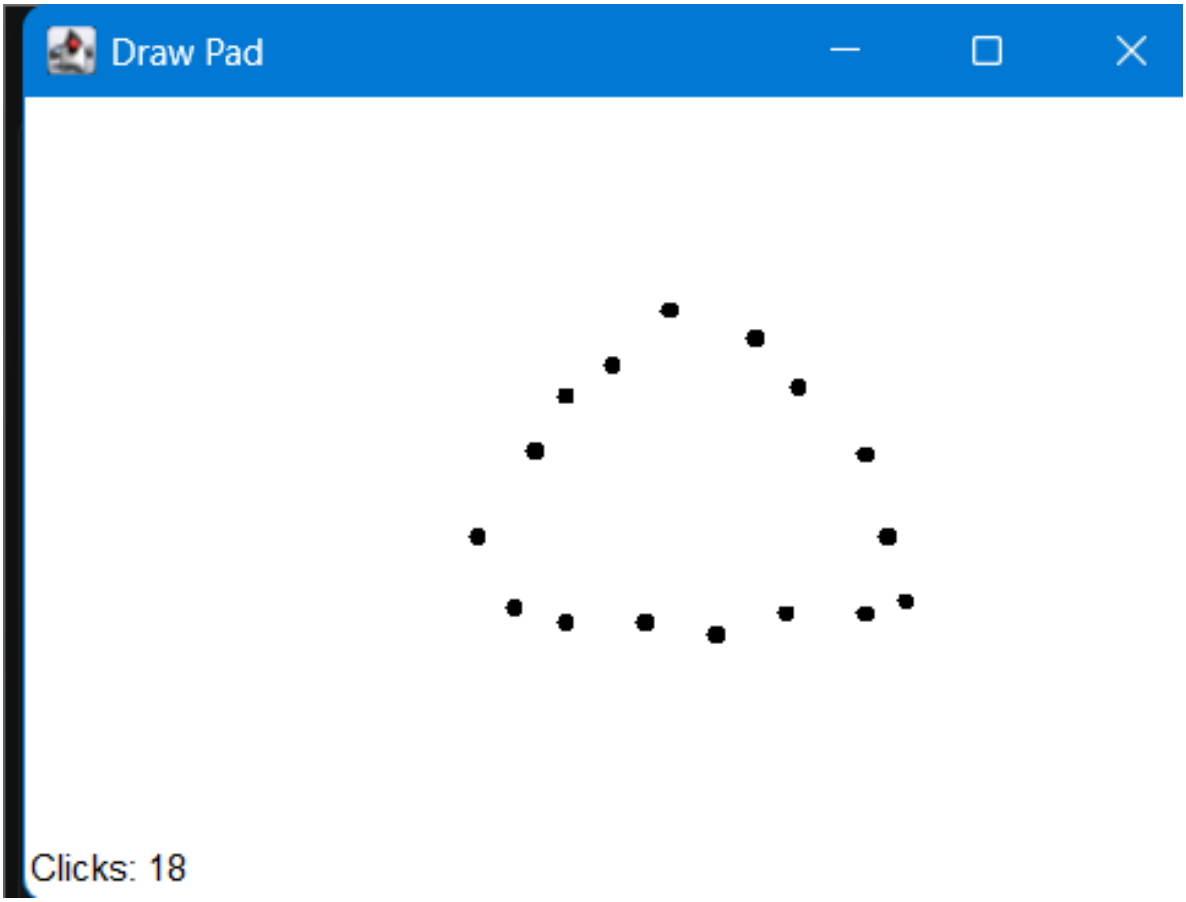
Task 2 – Appointment Desk service test



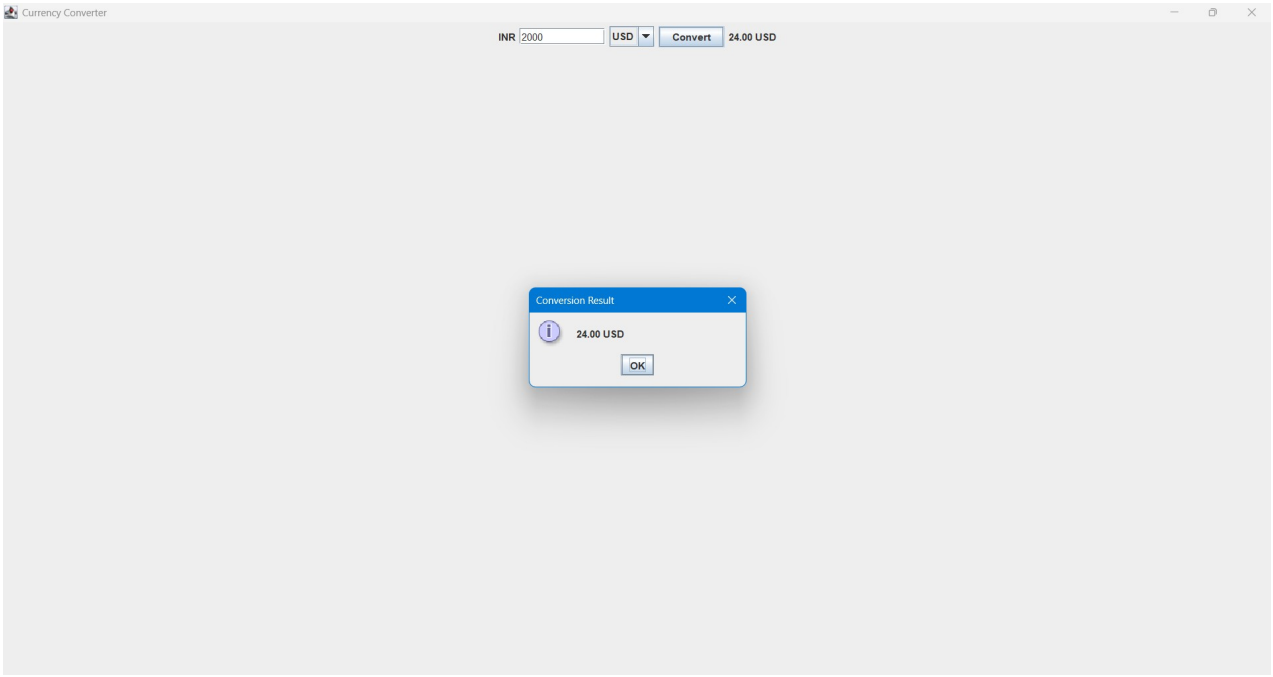
Task 3 – Campus Library Search



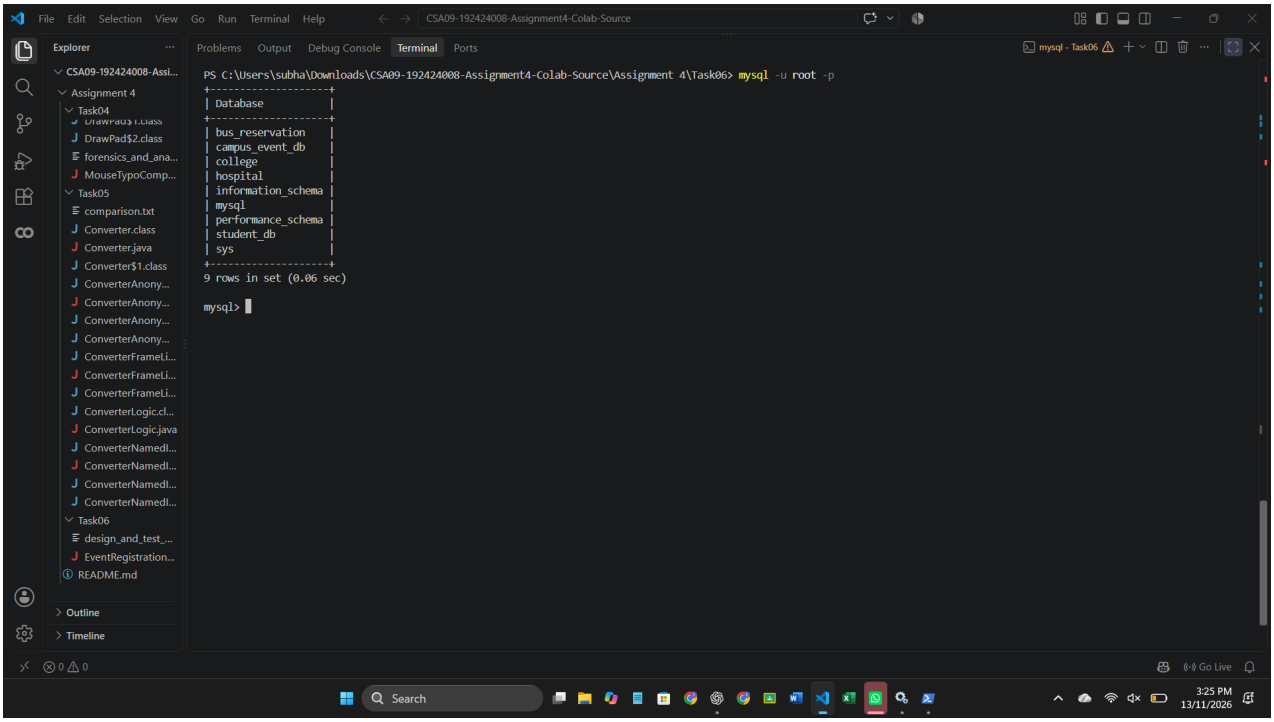
Task 4 – DrawPad (18 clicks)



Task 5 – Currency Converter result



Task 6 – MySQL database list



Task 6 – MySQL SHOW TABLES result

```
mysql> SHOW TABLES;
+---------------------------+
| Tables_in_campus_event_db |
+---------------------------+
| events                    |
| registrations             |
+---------------------------+
2 rows in set (0.03 sec)
```